

TITRE PROFESSIONNEL DEVELOPPEUR EN INTELLIGENCE ARTIFICIELLE
RNCP B3 - Simplon / ECE Paris

RAPPORT PROFESSIONNEL

Inflation Tracker France

Candidat	Alberto BONGUELE
Période	Juin - Aout 2026
Dépôt Git	https://github.com/GitAlberto/inflation-tracker
Branche	AlbertoFinB3 -> main
Soutenance	27 août 2026

SOMMAIRE

1. Contexte et problème adresse	3
2. Architecture globale.....	4
BLOC 1 : Collecte, stockage, mise a disposition (C1-C5)	5
C1 : Collecte de donnees multi-sources.....	5
C2 : Exploitation SQL des donnees	6
C3 : Pipeline d'agregation et nettoyage.....	7
C4 : Base de donnees PostgreSQL.....	7
C5 : API REST donnees	8
BLOC 2 : Modele IA, API modele, MLOps (C6-C13).....	9
C6 : Veille technologique	9
C7 : Benchmark des algorithmes	10
C8 : Entrainement du modele Prophet	11
C9 : API modele REST.....	13
C10 : Integration client-API.....	14
C11 : Monitoring du modele.....	14
C12 : Tests et couverture	16
C13 : CI/CD GitHub Actions.....	16
BLOC 3 : Application, CI/CD, monitoring applicatif (C14-C21)	18
C14 : Specifications fonctionnelles	18
C15 : Specifications techniques	18
C16 : Gestion de projet Agile (Kanban)	18
C17 : Application Streamlit	20
C18 : CI/CD application	23
C19 : Conteneurisation Docker.....	23
C20 : Monitoring applicatif	24
C21 : Gestion des incidents.....	25
3. Bilan et limites assumees	25

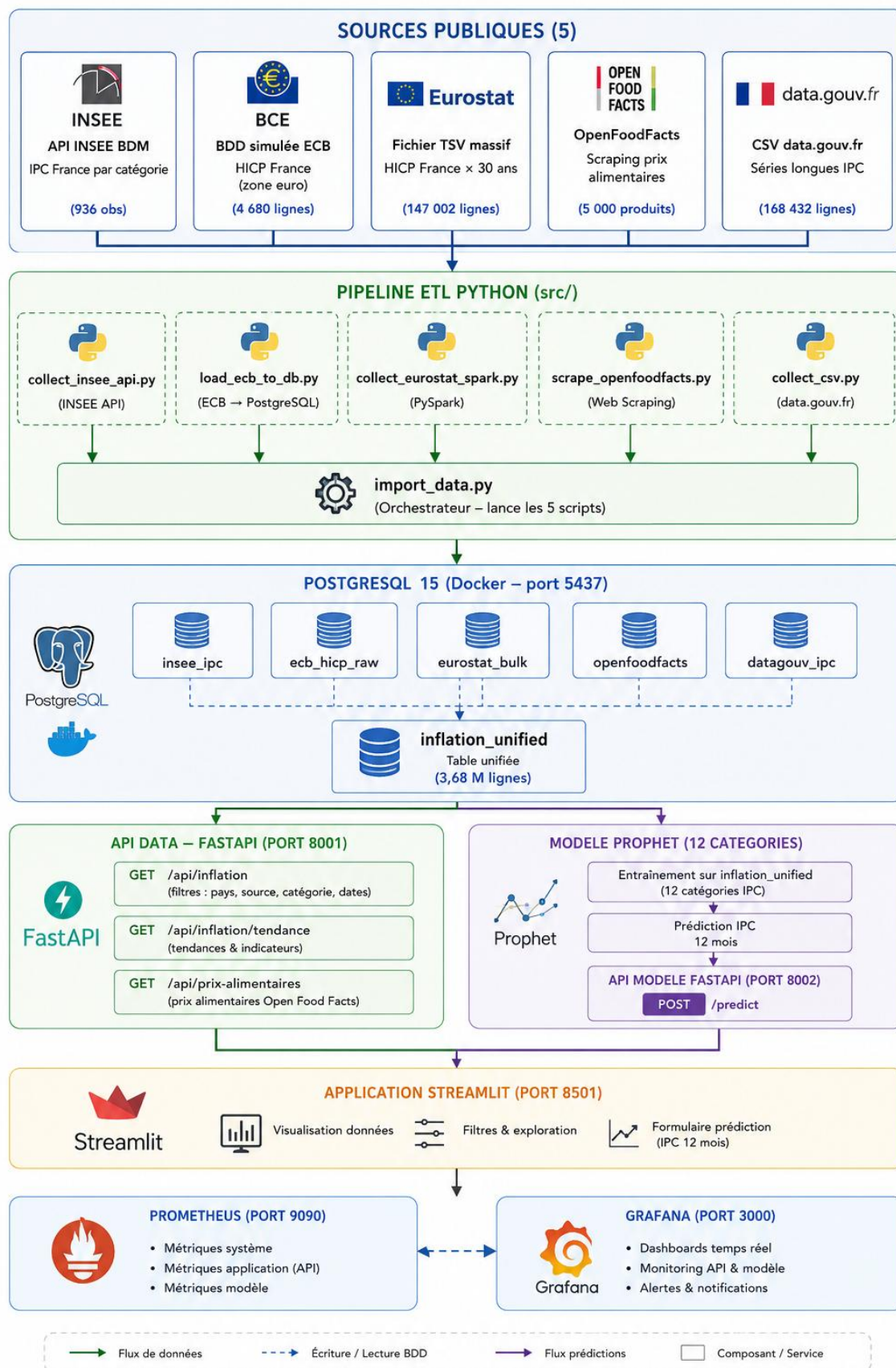
1. Contexte et problème adresse

Le kebab coûtait 3,50 euros en 2019. Il en coûte 7 euros en 2026. L'inflation est un phénomène que tout le monde subit mais que personne ne comprend vraiment. Les données publiques existent, publiées chaque mois par l'INSEE, la BCE, Eurostat, mais elles sont dispersées, au format technique (SDMX, CSV multicolonne, API REST avec codes COICOP), et inaccessibles pour un utilisateur non-statisticien.

Objectif du projet : concevoir et déployer un système complet de collecte, stockage, modélisation et visualisation de l'inflation en France, articulé en trois blocs techniques :

- **Bloc 1 :** Construire le socle data : collecter 5 sources hétérogènes, les normaliser et les exposer via une API REST.
- **Bloc 2 :** Entraîner un modèle de prédiction de l'IPC par catégorie COICOP, l'exposer via API, le monitorer.
- **Bloc 3 :** Développer une application Streamlit accessible à l'utilisateur final, avec CI/CD, monitoring applicatif et gestion des incidents.

2. Architecture globale



L'ensemble est orchestré par **start.sh** : **PostgreSQL** démarre en **Docker**, **Prometheus** et **Grafana** en **Docker**, les **APIs** et **Streamlit** en processus Python natifs.

BLOC 1 : Collecte, stockage, mise à disposition (C1-C5)

C1 : Collecte de données multi-sources

Ce que j'ai fait : cinq collecteurs Python distincts, chacun adapté à la nature de sa source.

Source	Script	Technologie	Volume
INSEE BDM	src/collect/collect_insee_api.py	API REST OAuth2	~13 séries mensuelles
data.gouv.fr	src/collect/collect_csv.py	CSV direct download	~180 k lignes
Open Food Facts	src/collect/scrape_openfoodfacts.py	Scraping BeautifulSoup	~5 k lignes
BCE	src/collect/load_ecb_to_db.py	API REST -> PostgreSQL	~50 k lignes
Eurostat	src/collect/collect_eurostat_spark.py	CSV bulk -> PySpark	3,51 M lignes

Incident rencontré : l'installation de PySpark a échoué avec « Failed to build wheel for pyspark », Java était absent du PATH de la session VS Code.

Résolution : installation d'**Eclipse Temurin JDK 21 (Adoptium)**, fermeture et réouverture de VS Code pour recharger les variables d'environnement.

```

((.venv) )
bongu@AckhilasProBook MINGW64 /c/Projets RNCP/B3 Projet Final (AlbertoFinB3)
$ pip install -r requirements.txt
warning: no files found matching '*.zip' under directory 'lib'
warning: no files found matching '*.pyi' under directory 'pyspark'
warning: no files found matching 'py.typed' under directory 'pyspark'
writing manifest file 'pyspark.egg-info\SOURCES.txt'
running build_scripts
creating build\scripts-3.12
error: [Errno 2] No such file or directory: 'pyspark\find_spark_home.py'
[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.
ERROR: Failed building wheel for pyspark
Failed to build pyspark

[notice] A new release of pip is available: 25.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
ERROR: Failed to build installable wheels for some pyproject.toml based projects (pyspark)
((.venv) )
bongu@AckhilasProBook MINGW64 /c/Projets RNCP/B3 Projet Final (AlbertoFinB3)
$

```

Résultat Eurostat : 3 511 040 lignes traitées par Spark, chargées dans eurostat_bulk en environ 6 minutes.

```

2026-07-04 10:33:26,544 [INFO] Résultat : 3511040 lignes x 5 colonnes
2026-07-04 10:33:31,172 [INFO] Données nettoyées sauvegardées : C:\Projets RNCP\B3 Projet Final\data\processed\bigdata_eurostat\eurostat_clean.csv
2026-07-04 10:33:32,645 [INFO] SparkSession arrêtée
2026-07-04 10:33:32,650 [INFO] Connexion à PostgreSQL (Docker port 5437)...
2026-07-04 10:33:33,030 [INFO] =====
2026-07-04 10:33:33,032 [INFO] ETAPE 3 – LOAD : insertion de 3511040 lignes dans eurostat_bulk
2026-07-04 10:39:59,061 [INFO] Vérification PostgreSQL : 3511040 lignes dans eurostat_bulk
2026-07-04 10:39:59,073 [INFO] =====
2026-07-04 10:39:59,073 [INFO] COLLECTE EUROSTAT TERMINÉE AVEC SUCCÈS
2026-07-04 10:39:59,073 [INFO] Raw      : data/raw/bigdata_eurostat/prc_hicp_manr_raw.tsv.gz
2026-07-04 10:39:59,073 [INFO] Processed: data/processed/bigdata_eurostat/eurostat_clean.csv
2026-07-04 10:39:59,076 [INFO] Base     : table eurostat_bulk dans PostgreSQL
2026-07-04 10:39:59,076 [INFO] =====
2026-07-04 10:40:02,674 [INFO] Closing down clientserver connection
Opération réussie : le processus de PID 31212 (processus enfant de PID 45868) a été arrêté.
Opération réussie : le processus de PID 45868 (processus enfant de PID 32904) a été arrêté.

```

Décision clé : toutes les sources sont filtrées sur géo=FR dès la collecte, évitant de stocker des données inutiles pour le périmètre France du projet.

C2 : Exploitation SQL des données

Ce que j'ai fait : deux fichiers de requêtes SQL documentées couvrant l'extraction et l'analyse.

- **src/sql/queries_extraction.sql** : extraction par source, jointures multi-sources, vérification de cohérence
- **src/sql/queries_analyse.sql** : tendances mensuelles, comparaisons inter-sources, détection d'anomalies

Exemple de requête analytique :

inflation_tracker/admin@INFLATION BASE* x
schema.sql x
inflation_tracker/admin@INFLATION BASE
No limit
Query
Query History
1 SELECT date_obs, categorie,
2 valeur,
3 valeur - LAG(valeur) OVER (PARTITION BY categorie ORDER BY date_obs) AS variation_mensuelle
4 FROM inflation_unified
5 WHERE source = 'INSEE' AND pays = 'FR'
6 ORDER BY categorie, date_obs;
7
Data Output
Messages
Notifications
SQL

	date_obs date	categorie character varying (100)	valeur numeric (10,4)	variation_mensuelle numeric
1	2015-01-01	00 - Ensemble	98.8600	[null]
2	2015-02-01	00 - Ensemble	99.5200	0.6600
3	2015-03-01	00 - Ensemble	100.1700	0.6500
4	2015-04-01	00 - Ensemble	100.2800	0.1100
5	2015-05-01	00 - Ensemble	100.5200	0.2400
6	2015-06-01	00 - Ensemble	100.4400	-0.0800
7	2015-07-01	00 - Ensemble	100.0300	-0.4100
8	2015-08-01	00 - Ensemble	100.3500	0.3200
9	2015-09-01	00 - Ensemble	99.9500	-0.4000
10	2015-10-01	00 - Ensemble	100.0100	0.0600

Chaque requête est commentée : pourquoi ce filtre, pourquoi cette jointure, ce que le résultat prouve.

C3 : Pipeline d'agrégation et nettoyage

Problème identifié : audit réalisé lors de l'intégration des sources : la colonne valeur contenait des grandeurs incomparables.

Source	Valeur brute	Problème
INSEE	119.17 (indice base 2015=100)	OK. Reference
DATAGOUV	Mélange indices + taux %	Filtre UNIT_MEASURE absent
ECB	2.4 (taux ANR %)	Pas un indice
EUROSTAT	3.2 (taux RCH_A %)	Mauvais dataset

Solution : règle unique : inflation_unified.valeur = indice IPC base 100 = 2015 pour toutes les sources. Filtres appliqués : UNIT_MEASURE=IX (DATAGOUV), dataset prc_hicp_midx avec I15 (Eurostat), série INX (ECB).

Colonne base_ref ajoutée : trace la base de référence de chaque source (2015 ou 2025 pour DATAGOUV rebases), permettant à l'application d'afficher un avertissement quand les bases sont incompatibles.

Script : src/aggregate/aggregate_clean.py

C4 : Base de données PostgreSQL

Schéma : 6 tables créées via src/database/schema.sql.

Table	Role
ecb_hicp_raw	Donnees brutes BCE (HICP zone euro)
eurostat_bulk	Donnees brutes Eurostat (PySpark)
datagouv_ipc	Donnees brutes data.gouv.fr
insee_ipc	Donnees brutes INSEE BDM
openfoodfacts	Prix alimentaires scrappe
inflation_unified	Table normalisee finale (toutes sources)

```

-- Nettoyage et le schéma existe déjà (pratique pour les tests)
11 DROP TABLE IF EXISTS inflation_unified CASCADE;
12 DROP TABLE IF EXISTS eurostat_bulk CASCADE;
13 DROP TABLE IF EXISTS datagouv_ipc CASCADE;
14 DROP TABLE IF EXISTS insee_ipc CASCADE;
15 DROP TABLE IF EXISTS ecb_hicp_raw CASCADE;
16
17
18
19 -- TABLE 1 : ecb_hicp_raw
20 -- Source : API ECB (Banque Centrale Européenne)
21 -- Contenu : HICP (Harmonised Index of Consumer Prices) zone euro, mensuel
22
23 CREATE TABLE ecb_hicp_raw (
24     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
25     time_period VARCHAR(7) NOT NULL, -- Format YYYY-MM ex: "2024-01"
26     obs_value NUMERIC(18, 4), -- Valeur de l'indice HICP
27     ref_year VARCHAR(18) NOT NULL, -- Année gdp ex: "192" (zone euro), "FR"
28 );
29
30 CREATE INDEX
31
32
33 Data Output Messages Notifications
34
35 NOTICE: table "inflation_unified" does not exist, skipping
36 NOTICE: la table "eurostat_bulk" n'existe pas, poursuite du traitement
37 NOTICE: la table "openfoodfacts" n'existe pas, poursuite du traitement
38 NOTICE: la table "datagouv_ipc" n'existe pas, poursuite du traitement
39 NOTICE: la table "insee_ipc" n'existe pas, poursuite du traitement
40 NOTICE: la table "ecb_hicp_raw" n'existe pas, poursuite du traitement
41
42 CREATE INDEX
43
44 Query returned successfully in 544 msec.
45
46 Total rows: Query complete 00:00:00.544
47
48

```

MCD Merise : modélise dans docs/mcd.md avec notation crow's foot (Mermaid erDiagram). 5 entités conceptuelles, 5 associations, cardinalités (1,1)->(0,N). Export PDF : docs/mcd_merise_2026-07-29.pdf.

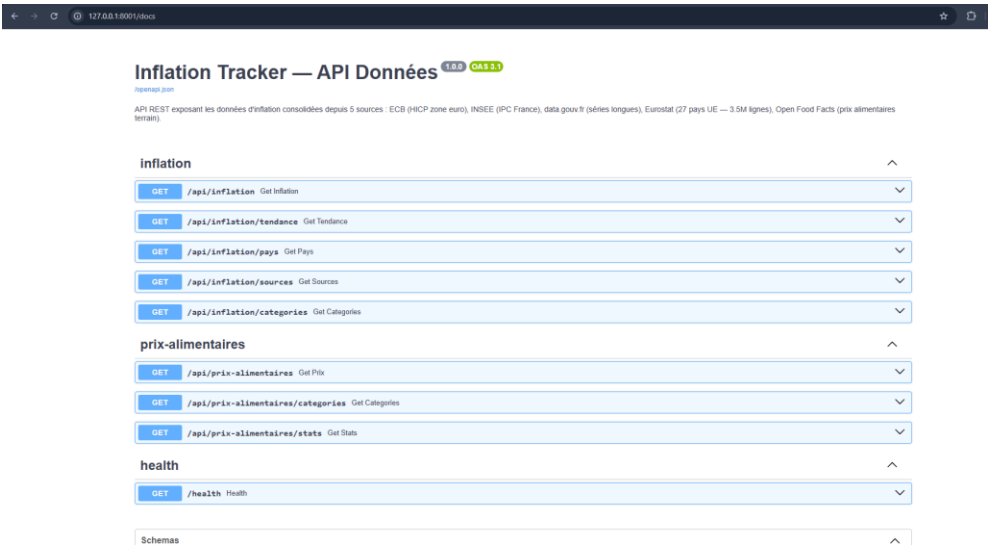
Registre RGPD : docs/registre_rgpd_inflation_tracker_fin.pdf. Aucune donnée personnelle collectée. Données publiques sous licences ouvertes (Licence Ouverte INSEE, CC BY Eurostat, ODbL OpenFoodFacts). Principe de minimisation appliqué.

Choix PostgreSQL : type NUMERIC(10,4) pour les valeurs IPC : aucune erreur d'arrondi flottant, critique pour des comparaisons de séries économiques sur 30 ans.

C5 : API REST données

Ce que j'ai fait : API FastAPI exposant inflation_unified avec **9 endpoints** documentés.

Endpoint	Description
GET /api/inflation	Données IPC avec filtres source/catégorie/date/pays
GET /api/inflation/tendance	Tendance mensuelle France
GET /api/inflation/pays	Codes pays disponibles (filtrable par source)
GET /api/inflation/sources	Sources disponibles
GET /api/inflation/categories	Catégories COICOP disponibles
GET /api/prix-alimentaires	Prix OpenFoodFacts
GET /api/prix-alimentaires/categories	Catégories alimentaires
GET /api/prix-alimentaires/stats	Statistiques prix
GET /health	Statut de l'API



Authentification : header X-API-Key obligatoire. Deux niveaux : clé admin (accès complet) et clé user (lecture publique). Implémente dans `api/data/auth.py`.

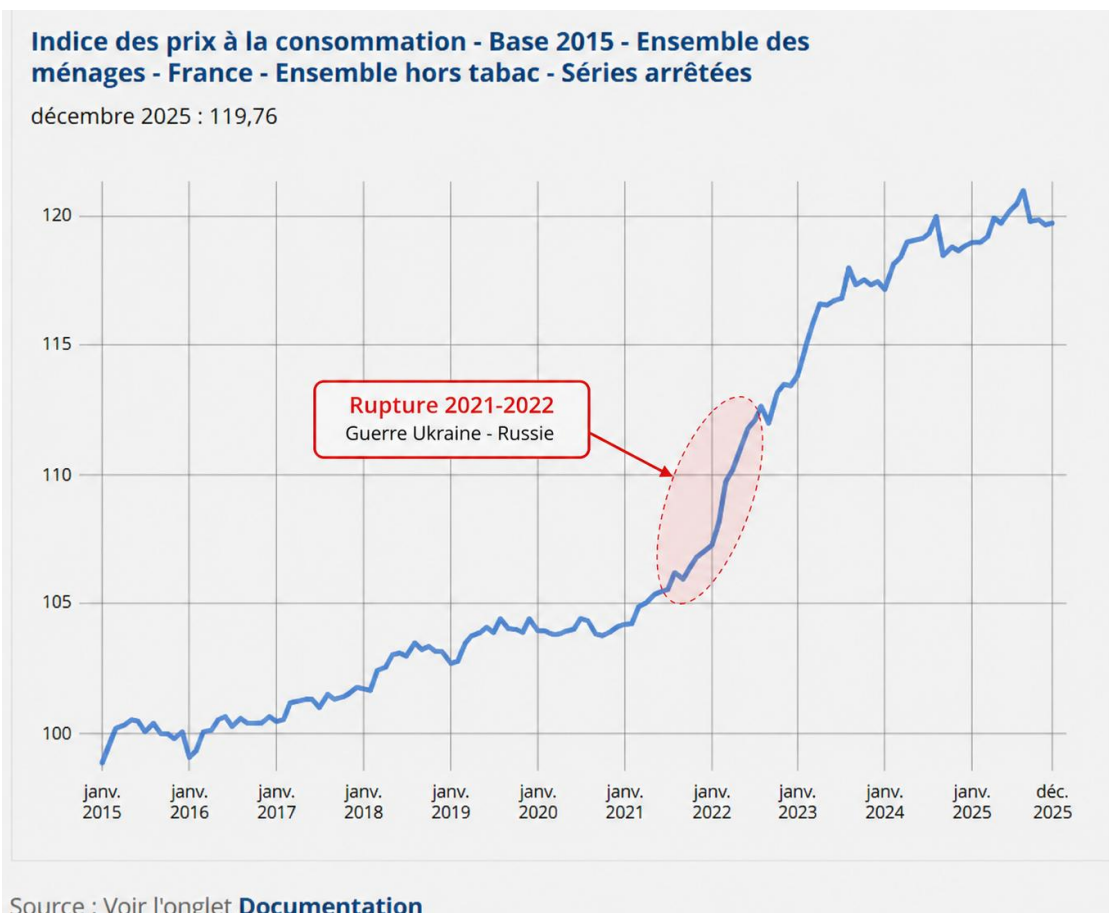
BLOC 2 : Modèle IA, API modèle, MLOps (C6-C13)

C6 : Veille technologique

Thématique : ML applique à la prédiction de séries temporelles économiques (IPC, inflation).

Sources consultées : publications BCE (ECB Working Papers), documents méthodologiques INSEE, articles NeurIPS/ICML sur séries temporelles, documentation Prophet (Meta AI Research).

Synthèse : trois familles de modèles évaluées (ARIMA, Prophet, LSTM). Contraintes identifiées pour l'inflation : saisonnalité mensuelle forte, ruptures structurelles ponctuelles (COVID, guerre Ukraine), volume de données limite (~60 points d'entraînement par catégorie). Prophet ressort comme le seul modèle gérant nativement ces trois contraintes.



Document complet : docs/veille_C6_final.pdf

C7 : Benchmark des algorithmes

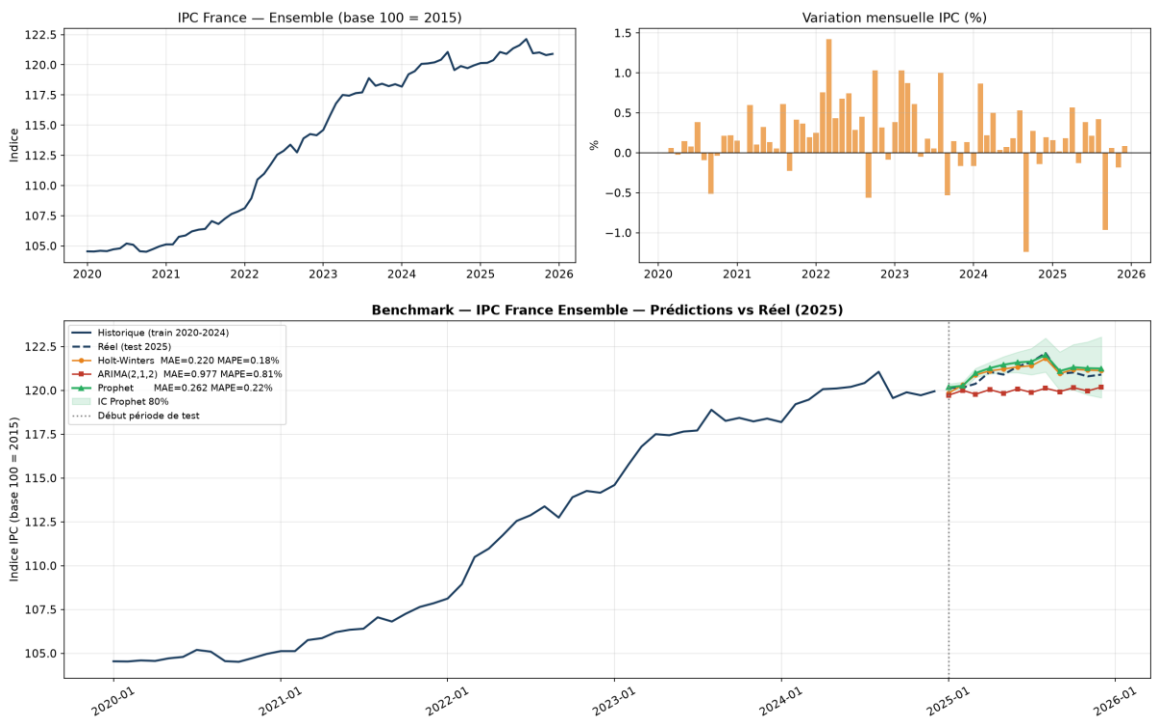
Modèles comparés : Prophet (Meta), ARIMA (2,1,2), Holt-Winters (lissage exponentiel triple).

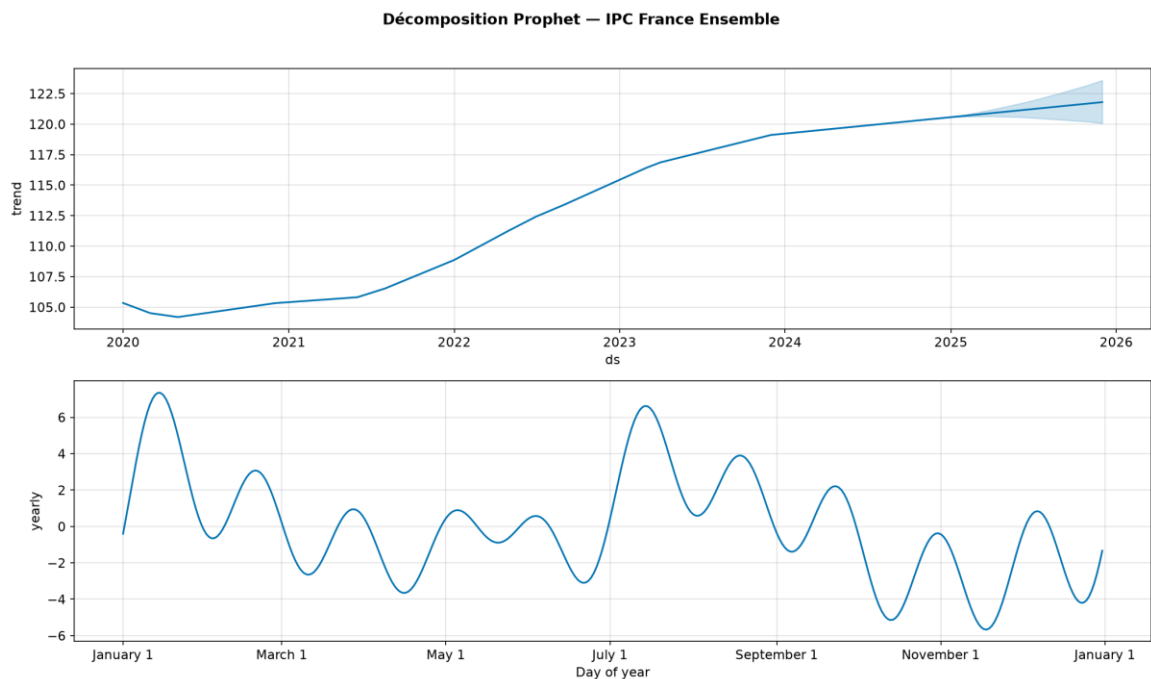
Protocole : entraînement sur IPC France Ensemble 2020-2024, évaluation sur 2025 (12 mois held-out). Même split pour les trois modèles.

Modèle	MAE	MAPE	Verdict
Holt-Winters	0.220	0.18%	Meilleur MAE
Prophet (retenu)	0.262	0.22%	Retenu
ARIMA(2,1,2)	0.977	0.81%	Insuffisant

Pourquoi Prophet et pas Holt-Winters malgré un MAE légèrement supérieur :

- Changepoints automatiques : détecte les ruptures structurelles (COVID 2020, guerre Ukraine 2022) sans re paramétrage manuel
- Intervalles de confiance natifs, requis pour l'affichage dans Streamlit
- Scalabilité : un même pipeline entraine 12 catégories sans ajuster les hyperparamètres





Résultats numériques : docs/benchmark_resultats.json

C8 : Entraînement du modèle Prophet

Ce que j'ai fait : 12 modèles Prophet entraînés, un par catégorie COICOP (00-11).

Hyperparamètres :

```
Prophet (
  yearly_seasonality=True,
  weekly_seasonality=False,
  daily_seasonality=False,
  changepoint_prior_scale=0.05
)
```

Split d'évaluation temporel strict :

- Train : de jan. 2020 à déc. 2024 (60 points)
- Eval : de jan. 2025 à déc. 2025 (12 points, jamais vus)

Un split temporel strict (pas de shuffle) est impératif pour les séries temporelles : mélanger des observations futures dans le train introduit du data leakage et surestime les performances.

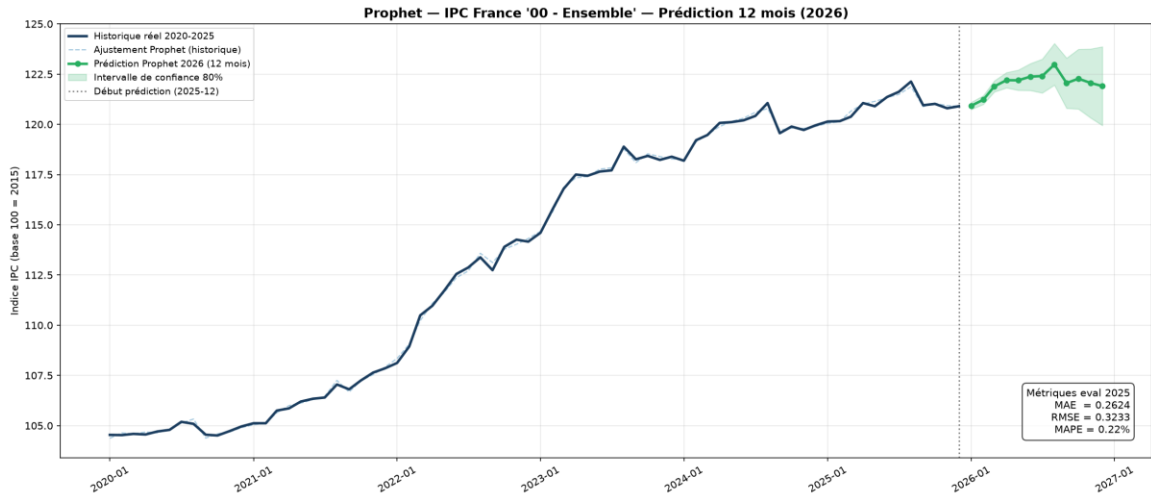
Métriques Prophet par catégorie COICOP : Evaluation 2025 (n_eval=12, n_train=60)

Tri par MAE croissant. Vert : MAE <= 1.0 pt IPC. Jaune : MAE > 3.0 pts IPC.

Categorie	MAE	RMSE	MAPE
00 - Ensemble	0.26	0.32	0.22%
05 - Ameublement, équipement ménager	0.28	0.35	0.25%
10 - Enseignement	0.42	0.54	0.35%
07 - Transports	0.72	1.04	0.57%
11 - Restaurants et hôtels	0.73	0.87	0.58%
03 - Habillement et chaussures	1.10	1.28	1.05%
01 - Alimentation et boissons	1.51	1.73	1.11%
09 - Loisirs et culture	1.43	1.48	1.30%
06 - Sante	1.52	1.53	1.61%
02 - Boissons alcoolisées et tabac	1.76	1.80	1.14%
08 - Communications	3.79	4.94	4.93%
04 - Logement, eau, gaz, électricité	4.83	5.09	3.69%
Moyenne (12 catégories)	1.43	1.62	1.31%

Les catégories **08 (Communications)** et **04 (Logement/énergie)** présentent des MAE élevés : chocs réglementaires et choc Ukraine sont des ruptures structurelles difficiles à prédire pour un modèle entraîné sur 60 points.

Catégorie 12 exclue : la série INSEE BDM de la catégorie 12 (Biens et services divers : coiffure, assurances, frais bancaires) n'a pas été intégrée dans le collecteur. De plus, ses mécanismes de formation de prix sont structurellement différents des autres catégories.



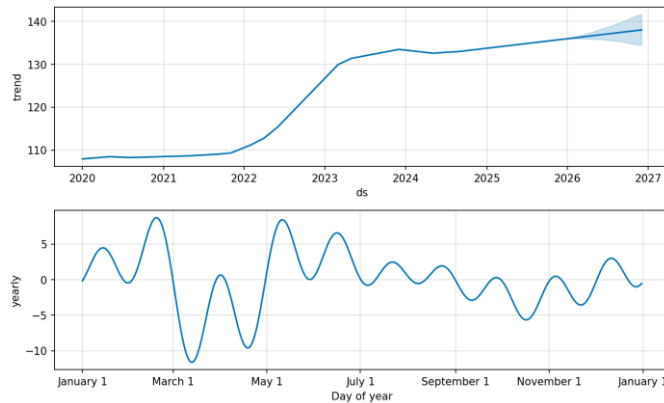
```
bongu@AckhilasProBook MINGW64 /c/Projets RNCP/B3 Projet Final/model (AlbertoFinB3)
$ python evaluate.py
```

```
=====
EVALUATION C8 - Prophet IPC France (inflation-tracker)
=====
```

```
=====
MÉTRIQUES C8 - Prophet IPC France x 13 catégories (eval 2025)
=====
```

Catégorie	MAE	RMSE	MAPE%
Ensemble hors énergie	0.2330	0.2900	0.20%
00 - Ensemble	0.2624	0.3233	0.22%
05 - Meubles, articles de ménage et entr	0.2848	0.3455	0.25%
10 - Enseignement	0.4223	0.5381	0.35%
07 - Transports	0.7174	1.0428	0.57%
11 - Restaurants et hôtels	0.7321	0.8677	0.58%
03 - Articles d'habillement et chaussure	1.0987	1.2754	1.05%
09 - Loisirs et culture	1.4327	1.4783	1.30%
01 - Alimentation et boissons non alcool	1.5058	1.7286	1.11%
06 - Santé	1.5209	1.5281	1.61%
02 - Boissons alcoolisées, tabac et stup	1.7631	1.8008	1.14%
08 - Communications	3.7871	4.9403	4.93%
04 - Logement, eau, gaz, électricité et	4.8327	5.0883	3.69%
Moyenne (13 catégories)	1.4302		1.31%

Décomposition Prophet — IPC Alimentation France (INSEE 2020-2025)
Tendance - Saisonnalité annuelle



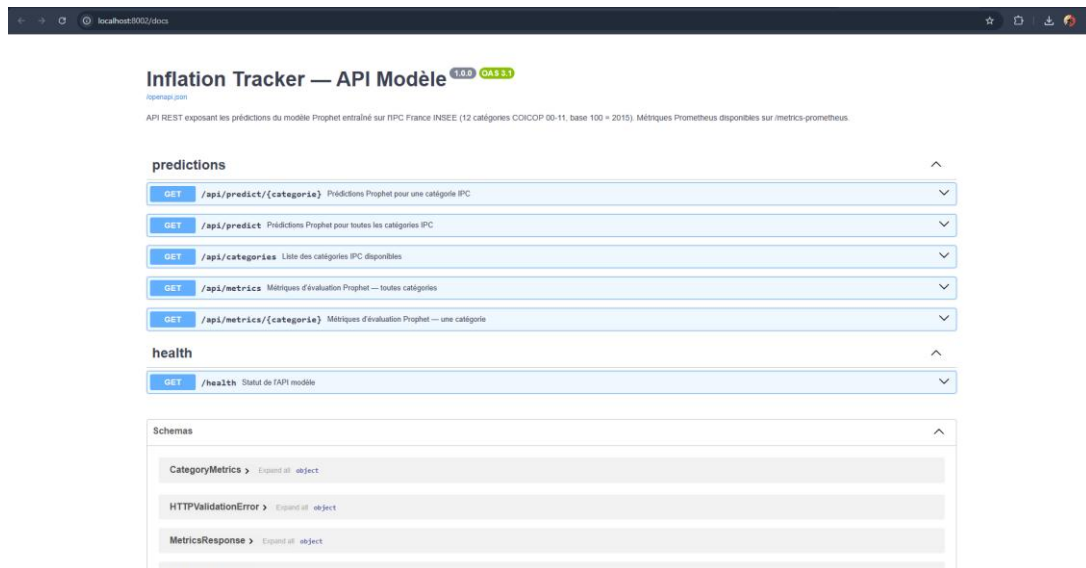
Scripts : model/train.py, model/evaluate.py.

Métriques : model/metrics.json

C9 : API modèle REST

Ce que j'ai fait : API FastAPI exposant les 12 modèles Prophet avec 6 endpoints.

Endpoint	Description
GET /api/predict/{categorie}	Prédiction Prophet pour une catégorie, horizon paramétrable
GET /api/predict	Prédictions toutes catégories
GET /api/categories	Liste des 12 catégories disponibles
GET /api/metrics	MAE/RMSE/MAPE toutes catégories
GET /api/metrics/{categorie}	Métriques d'une catégorie
GET /health	Statut de l'API modele



Séparation des APIs justifiée : cycle de vie différent (données **stables** vs **modèles** re entraînaables), scalabilité indépendante, métriques Prometheus distinctes par API.

C10 : Intégration client-API

Ce que j'ai fait : le client Streamlit (app/api_client.py) consomme l'API modèle avec authentification X-API-Key. Deux niveaux d'accès implémentés dans api/model/auth.py :

- verify_user_key -> accepte API_KEY ou API_KEY_USER, routes publiques (/predict/{categorie}, /categories)
- verify_admin_key -> accepte uniquement API_KEY, routes sensibles (/metrics, /predict toutes catégories)

Incident postmortem : lors du déploiement Docker, le client Streamlit envoyait la bonne clé mais l'API répondait 403. Cause racine : load_dotenv(override=True) dans auth.py écrasait la variable API_KEY injectée par Docker avec la valeur du fichier .env.local. Correction : override=False + bloc environnement dans docker-compose.yml.

(Postmortem complet : docs/incident_2026-07-25.md)

C11 : Monitoring du modèle

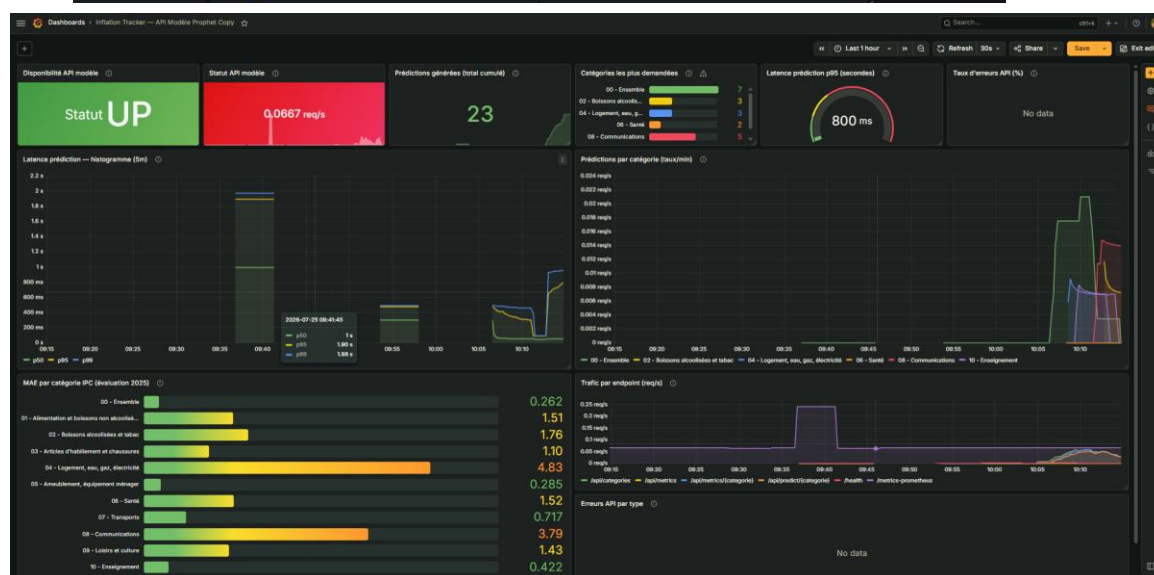
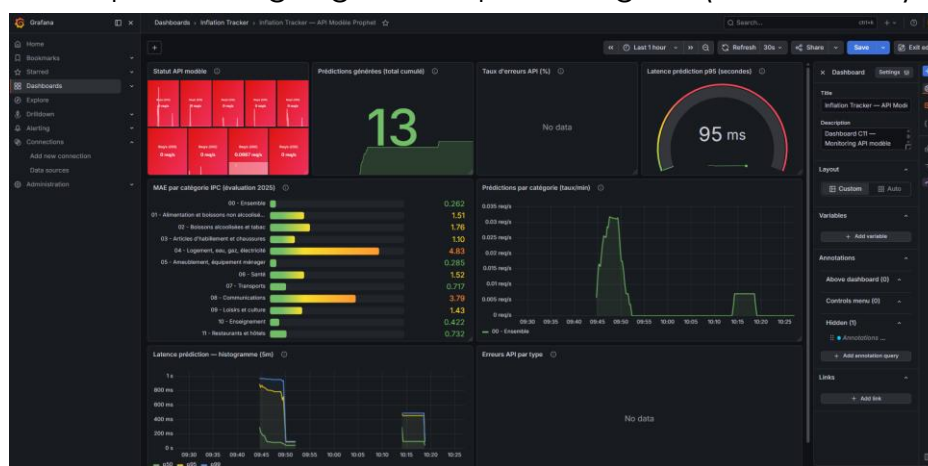
Stack : Prometheus scrape l'API modèle toutes les 15 secondes via /metrics-prometheus. Grafana lit Prometheus et affiche le dashboard en temps réel (refresh 30s).

Métriques exposées :

Métrique Prometheus	Type	Ce qu'elle mesure
<code>http_requests_total</code>	Counter	Requêtes par endpoint et code HTTP
<code>http_request_duration_seconds</code>	Histogram	Latence p50/p95/p99
<code>predictions_total</code>	Counter	Prédictions par categorie
<code>api_up</code>	Gauge	Disponibilité de l'API

Dashboard Grafana

(monitoring/grafana/dashboards/model_dashboard.json), 11 panneaux :
 Stat : disponibilité, req/s, prédictions totales, top catégories, latence p95, taux erreurs. Timeseries : latence p50/p95/p99, prédictions par catégorie, trafic par endpoint.
 Bar gauge : MAE par categorie (toutes les 12).



Problème de synchronisation résolu : le monitoring ne se mettait pas à jour en temps réel car les containers Docker `api_model` et `api_data` occupaient les ports 8001/8002, empêchant les processus natifs de `start.sh` de se lier. Prometheus scrapait alors le mauvais processus. Correction dans `start.sh` : arrêt des containers API Docker avant le démarrage natif, démarrage de Prometheus avec `--no-deps`.

C12 : Tests et couverture

47 tests, couverture 93%

Fichier de test	Ce qui est testé
tests/test_collect.py	Collecteurs : retournent un DataFrame non vide, colonnes attendues
tests/test_aggregate.py	Pipeline : pas de nulls sur colonnes clés, normalisation COICOP
tests/test_api_data.py	Endpoints data : 200/401/403/422, pagination, filtres
tests/test_model.py	Slugify, compute_metrics, format metrics.json
tests/test_api_model.py	Endpoints modèle : health, catégories, metrics, 404, 422

```

----- coverage: platform win32, python 3.12.10-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
api\___init__.py                     0      0  100%
api\data\___init__.py                0      0  100%
api\data\auth.py                     9      1   89%    37
api\data\database.py                18      0  100%
api\data\main.py                     11      0  100%
api\data\routes\___init__.py         0      0  100%
api\data\routes\inflation.py        66      4   94%    87-88, 129, 166
api\data\routes\prix.py              28      1   96%    98
api\data\schemas.py                 28      0  100%
api\model\___init__.py               0      0  100%
api\model\auth.py                   12      1   92%    42
api\model\main.py                    48      8   83%    59-64, 133, 171
api\model\metrics.py                  6      0  100%
api\model\routes\___init__.py        0      0  100%
api\model\routes\predict.py          56      7   88%    55, 121-123, 149-150, 153, 176
api\model\schemas.py                 23      0  100%
-----
TOTAL                               305     22   93%

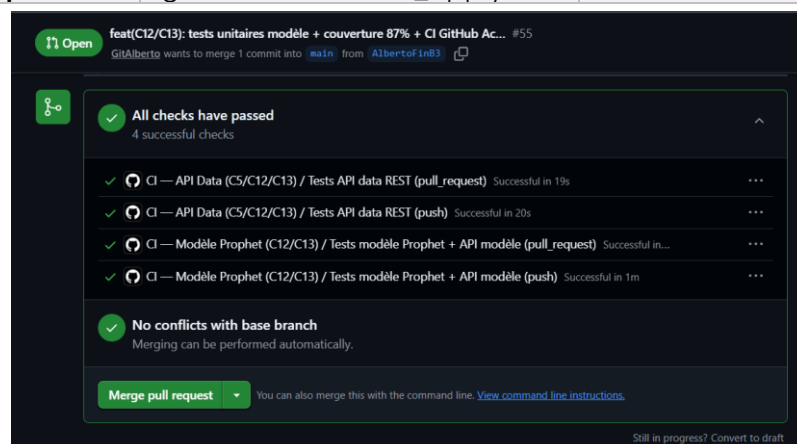
```

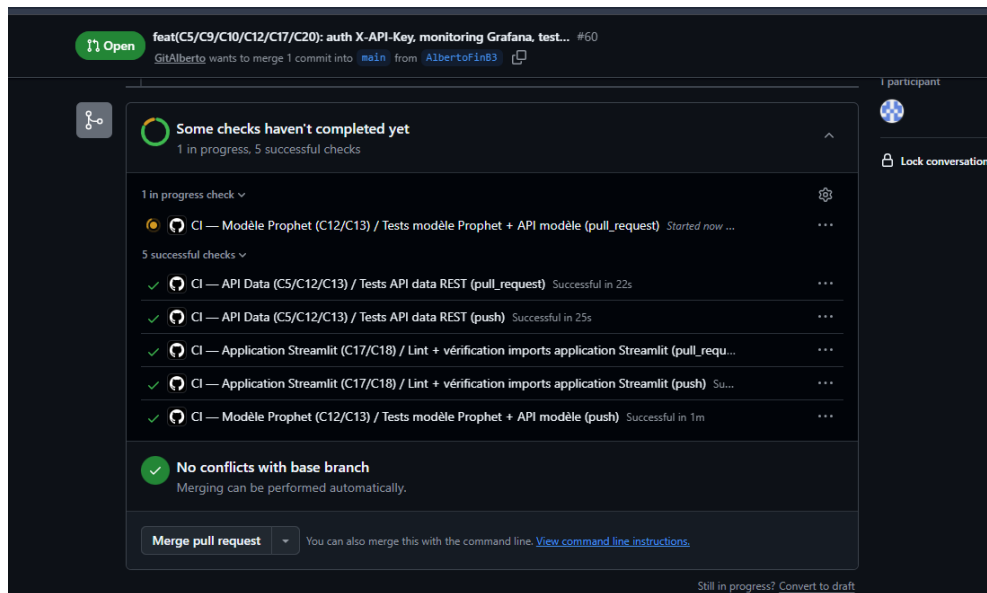
Stratégie CI : les tests nécessitant les .pkl ou PostgreSQL sont marqués `@pytest.mark.skipif(os.getenv("CI") == "true")` pour s'exécuter uniquement en local. En CI, seuls les tests unitaires purs tournent, aucune dépendance externe requise.

C13 : CI/CD GitHub Actions

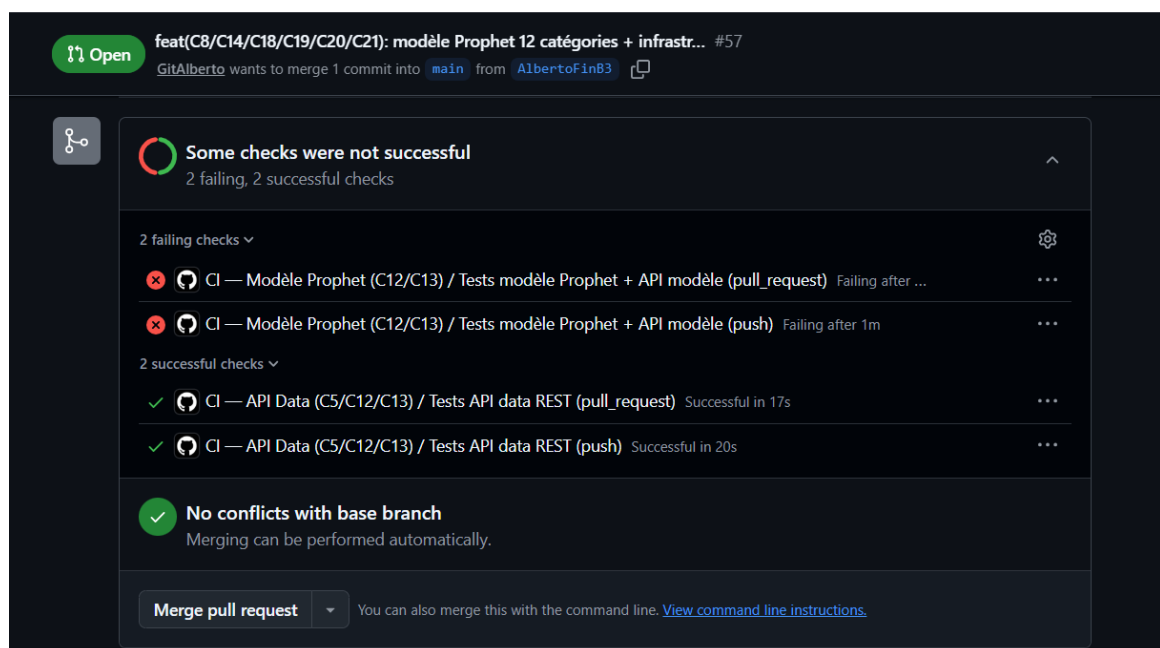
Trois pipelines déclenchés à chaque push et pull request :

Workflow	Fichier	Ce qu'il exécute
CI Bloc 1 Data	.github/workflows/ci_data.yml	Tests API data, ~20s
CI Bloc 2 modèle	.github/workflows/ci_model.yml	Tests modèle + API modèle, ~1min
CI Bloc 3 App	.github/workflows/ci_app.yml	Lint + vérification imports Streamlit





Incident CI documenté : PR #57 : 2 checks échouent sur CI modèle.
Cause : les tests attendaient 13 catégories, le modèle n'en entraîne que 12. Commit correctif : fix(C12/C13): test_list_available 13->12 catégories COICOP. CI repasse au vert.



Réentraînement : non automatisé en CI. L'entraînement Prophet nécessite CmdStan (~500 Mo, ~15 min). Le fichier model/metrics.json est versionné (trace les performances), les .pkl sont dans .gitignore (binaires lourds, régénérables).

BLOC 3 : Application, CI/CD, monitoring applicatif (C14-C21)

C14 : Spécifications fonctionnelles

Document : docs/specs_fonctionnelles.md, 9 user stories, 32 critères d'acceptation.

Couverture : 28/32 critères implémentés. Les 4 restants sont documentés avec une note de périmètre :

- **Sélection de pays** : périmètre fixe a France (toutes sources filtrées geo=FR à la collecte)
- Source par défaut : INSEE sélectionnée par défaut (source officielle de référence)
- **Note base 2025 visible dans l'UI** : documente dans specs_techniques, non affiché dans Streamlit
- **Barres 3 couleurs** : 2 couleurs implémentées (vert ≤ 5 pts / rouge > 5 pts)

MCD : docs/mcd.md, modele Merise avec 5 entités conceptuelles (SOURCE, CATEGORIE_COICOP, PAYS, OBSERVATION_IPC, PRIX_ALIMENTAIRE), 5 associations avec cardinalités (1,1)->(0,N).

C15 : Spécifications techniques

Document : docs/specs_techniques.md, 6 décisions techniques documentées avec justification.

Décision	Résumé
#1 Normalisation	Règle unique : indice base 100=2015 pour toutes les sources
#2 API séparées	API data (8001) et API modele (8002) : cycles de vie différents
#3 Prophet	Changepoints + IC natifs + scalabilité. Voir C7
#4 Split temporel strict	Pas de shuffle, évite le data leakage sur séries temporelles
#5 Exclusion categorie 12	Serie INSEE absente + hétérogénéité des prix divers
#6 Réentraînement manuel	CmdStan incompatible avec CI automatisé

C16 : Gestion de projet Agile (Kanban)

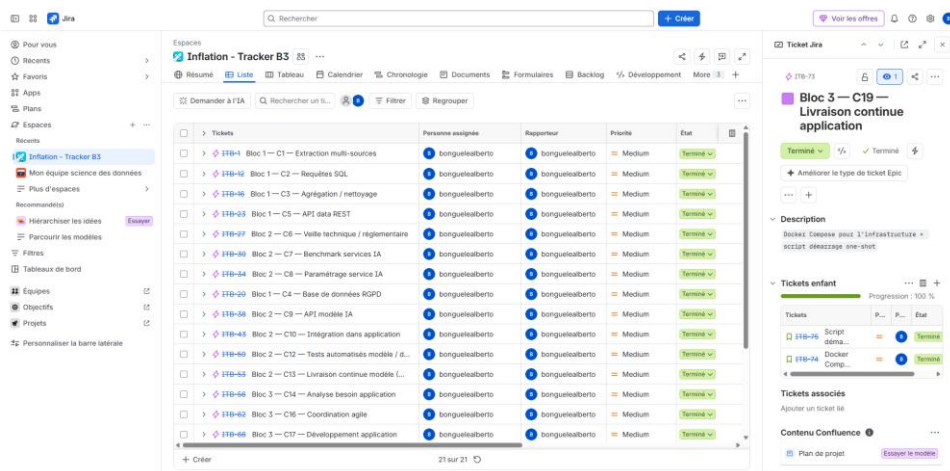
Méthode : Kanban (projet solo, périmètre évolutif, flux continu par compétence).

Outil : Jira — board 4 colonnes (Backlog / In Progress / Review / Done).

Three screenshots of the Jira interface showing the 'Inflation - Tracker B3' project. The first screenshot shows a list of tickets with the 'À faire' status. The second screenshot shows the same list with some tickets marked 'Terminé' and one 'En cours'. The third screenshot shows the list with all tickets marked 'Terminé'. Each screenshot includes a sidebar with navigation options and a right-hand panel showing the details of a specific ticket, 'Correction et test de non régression'.

The Jira interface displays the 'Inflation - Tracker B3' project. The main view is a list of tickets, showing columns for 'Personne assignée', 'Rapporteur', 'Priorité', and 'État'. The tickets are listed in a table format, with the 'État' column showing various statuses like 'À faire', 'Terminé', and 'En cours'.

The right-hand panel shows the details of a specific ticket, 'Correction et test de non régression'. It includes a description, a list of sub-tasks, and a section for 'Contenu Confluence'.



C17 : Application Streamlit

Ce que j'ai fait : application 4 pages avec thème financier (app/theme.py).

Page	Contenu
Accueil	KPIs IPC 2022-2025, statut APIs, navigation
Analyse Historique	Graphique IPC multi-sources, slider periode, multi-catégories (max 5), stats descriptives
Analyse par Catégorie	IPC + variation YoY, heatmap mensuelle, décomposition saisonnière
Prédictions Prophet	Courbe historique + prédiction + IC 80%, MAE/RMSE/MAPE en sidebar, tableau détaillé
Métriques Modèle	Bar chart MAE 12 catégories, scatter RMSE vs MAE, tableau complet

La page d'accueil : présente un tableau d'impact concret : pour cinq niveaux de budget représentatifs (300 €, 500 €, 800 €, 1 200 €, 2 000 €/mois), l'application calcule le budget équivalent en euros constants sur la période, ainsi que la hausse mensuelle et annuelle correspondante.

Un tableau de calcul pas à pas (hausse IPC → % → nouveau budget → hausse en €) rend la mécanique lisible pour un utilisateur non initié à l'économie.

La page Prédictions : intègre un simulateur budgétaire permettant à l'utilisateur de saisir son budget mensuel actuel et d'en estimer l'évolution selon les prédictions Prophet.

La formule appliquée est :

$$\text{budget_futur} = \text{montant} \times (\text{IPC_prédit} / \text{IPC_actuel})$$

La base de référence de l'indice (2015) s'annule dans le ratio : seule la variation relative compte.

L'interface expose cinq indicateurs en ligne :

- IPC actuel
- IPC prédit à l'horizon choisi
- Budget actuel
- Variation estimée (%)
- Budget projeté

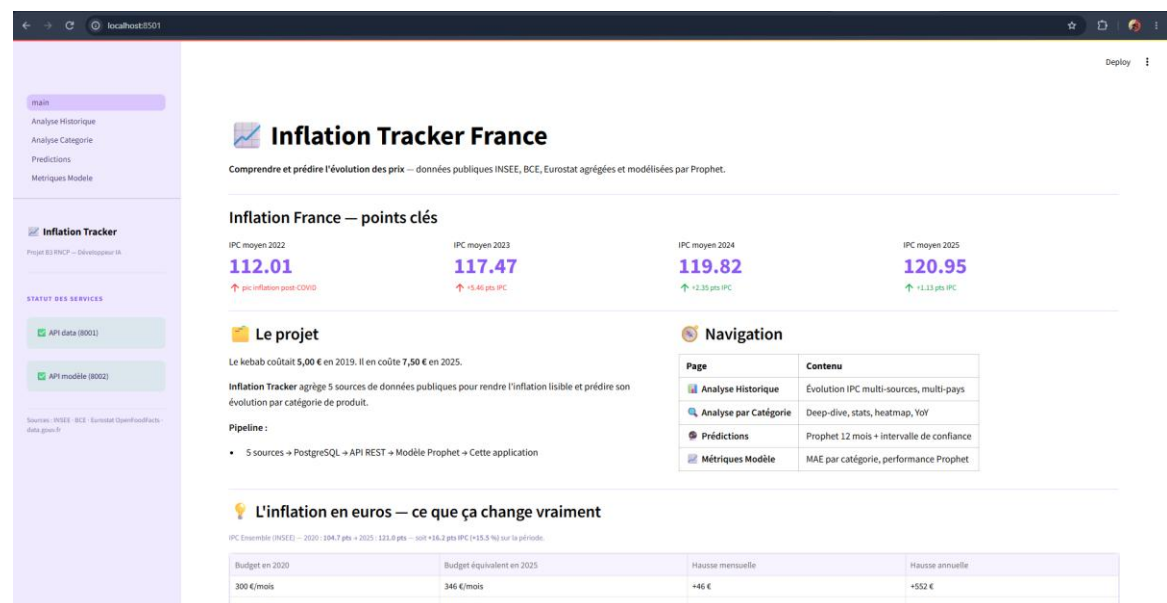
Ainsi qu'un intervalle de confiance à 80 % issu du modèle.

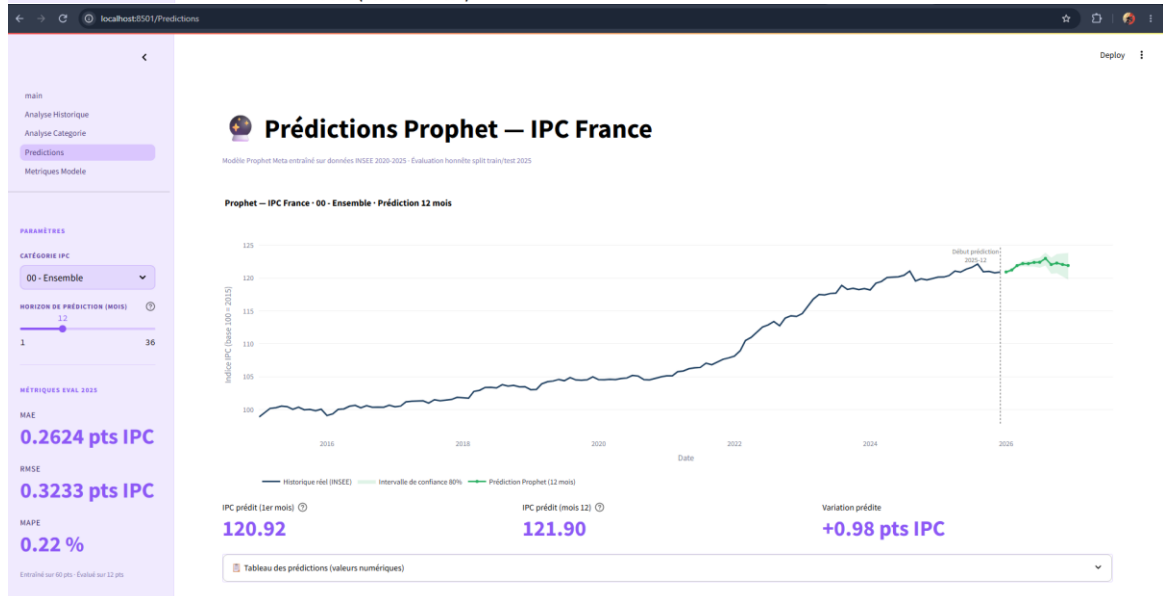
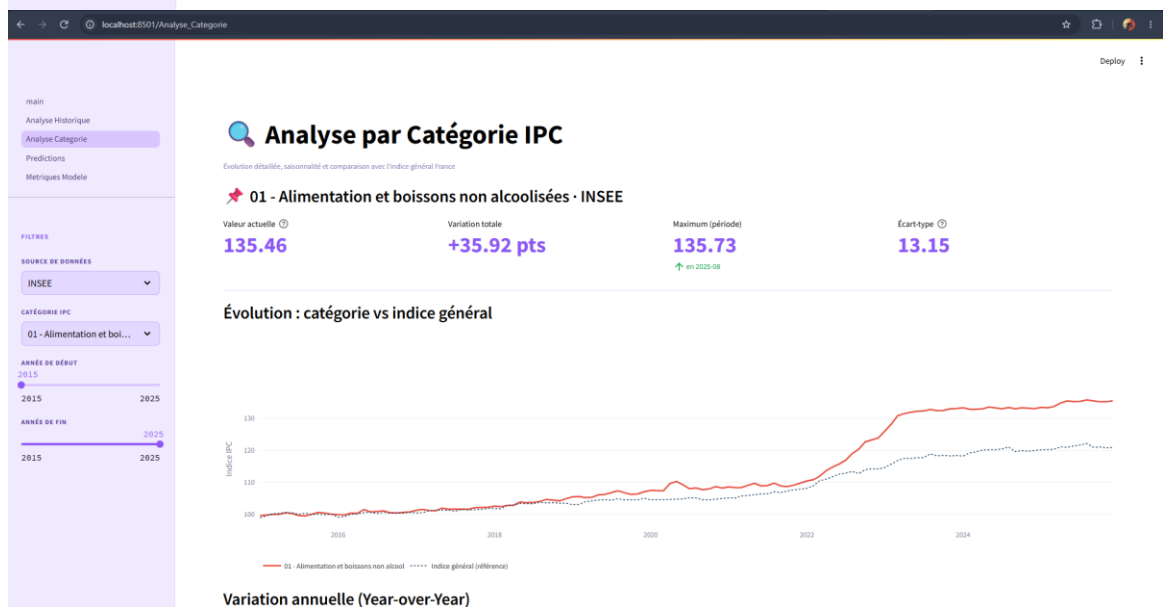
La variation en pourcentage est calculée directement sur le ratio IPC, et non sur les budgets arrondis, pour éviter une distorsion sur les petits montants.

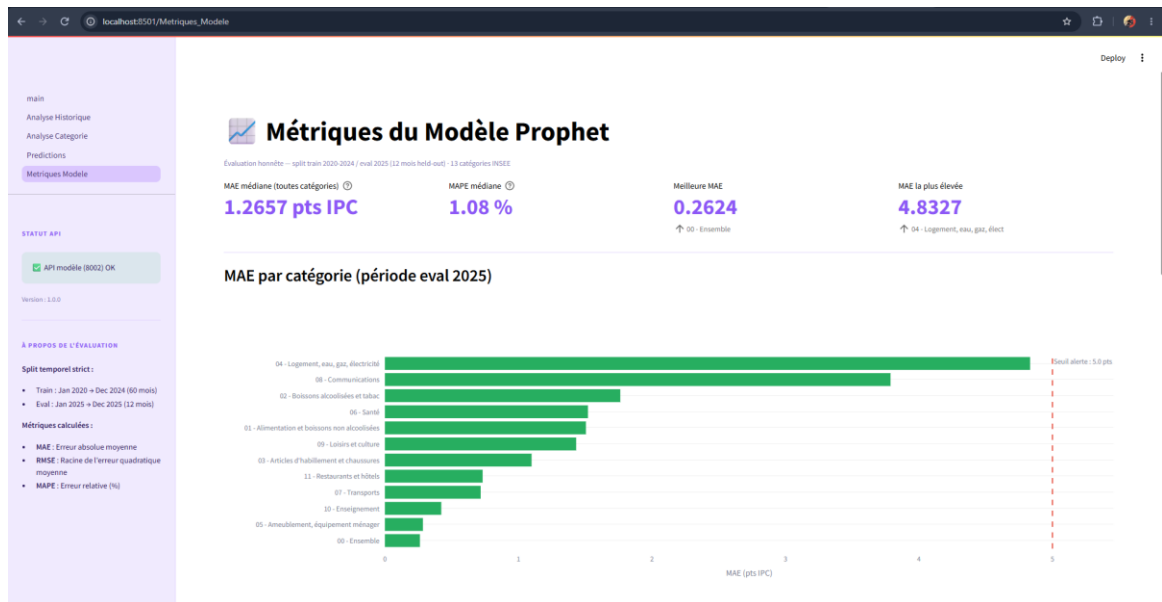
La page Métriques Modèle : présente les erreurs MAE et RMSE sous forme de barres groupées horizontales, triées par MAE croissante. Lorsque la barre RMSE (orange) dépasse nettement la MAE (violet), cela signale des erreurs ponctuelles importantes, typiquement des chocs d'inflation difficiles à anticiper pour un modèle de série temporelle.

Cette représentation remplace un nuage de points moins lisible, et rend immédiatement visible l'écart RMSE/MAE par catégorie.

Performances : données historiques en cache 5 min (@st.cache_data(ttl=300)), prédictions en cache 1 min (ttl=60).
Chargement < 3s pour les données, < 15s pour les prédictions Prophet.







C18 : CI/CD application

Le pipeline `.github/workflows/ci_app.yml` exécute à chaque push :

- Lint Flake8 de l'application Streamlit
- Vérification des imports (aucune dépendance cassée)
- Pas de test end-to-end. Streamlit nécessite un navigateur headless non disponible en CI standard.

C19 : Conteneurisation Docker

`docker-compose.yml`, 4 services orchestrés :

Service	Image	Port	Rôle
postgres	postgres:15	5437	Base de données PostgreSQL
prometheus	prom/prometheus	9090	Collecte des métriques
grafana	grafana/grafana	3000	Dashboards monitoring
api_data	build local	8001	API data (Docker uniquement)
api_model	build local	8002	API modèle (Docker uniquement)

Décision architecturale : en production (démon), les APIs et Streamlit tournent en processus Python natifs (`.venv`) pour éviter les délais de rebuild Docker à chaque modification. Prometheus et Grafana restent en Docker. Cette architecture hybride est documentée dans start.sh.

C20 : Monitoring applicatif

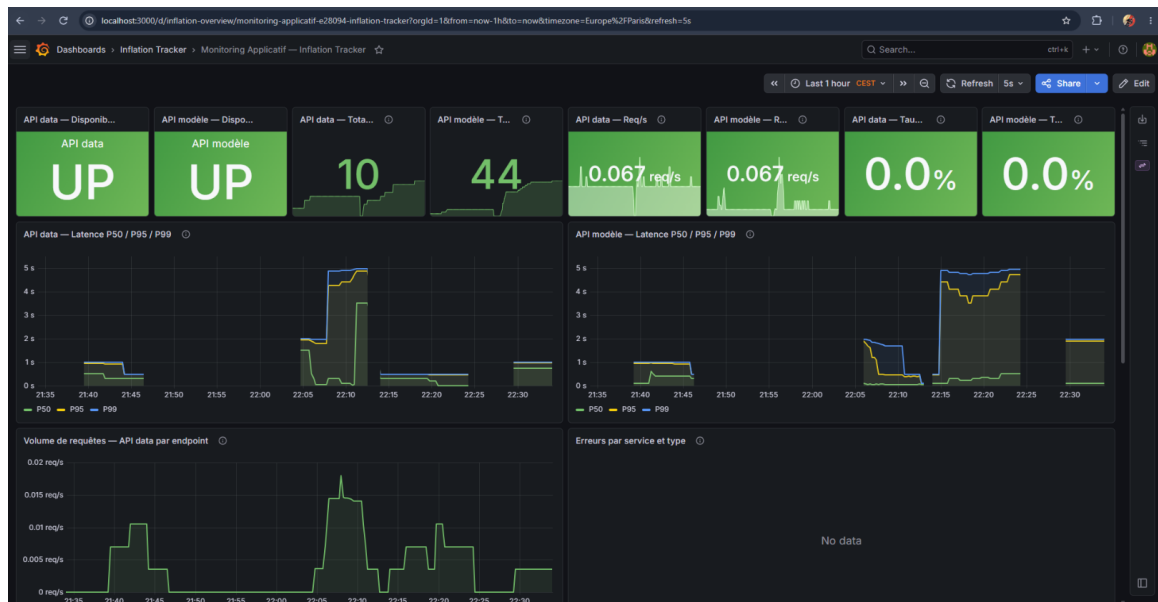
Dashboard Grafana provisionne automatiquement au démarrage (monitoring/grafana/provisioning/).

```

1 groups:
2   - name: inflation_tracker_model
3     # Règles d'alertes pour l'API modèle Prophet (C11)
4     # Prometheus évalue ces règles toutes les 15 secondes (evaluation_interval dans prometheus.yml)
5     rules:
6
7     # Alerte 1 – Dérive du modèle : MAE trop élevée
8     # Déclenchée si la MAE d'une catégorie dépasse 5 points IPC pendant 5 minutes
9     # Seuil : 5 pts est raisonnable sur un IPC base 100 (~5% d'erreur relative)
10    - alert: ModelMAETropElevee
11      expr: inflation_prediction_mae > 5
12      for: 5m
13      labels:
14        severity: warning
15      annotations:
16        summary: "Dérive modèle Prophet – MAE élevée"
17        description: >
18          La MAE de la catégorie {{ $labels.categorie }} est de {{ $value | printf "%.2f" }}
19          depuis plus de 5 minutes (seuil : 5 pts IPC).
20          Action : vérifier les nouvelles données INSEE et réentraîner le modèle.
21
22    # Alerte 2 – Latence élevée sur les prédictions
23    # p95 de la latence > 10 secondes sur 2 minutes → charge anormale
24    - alert: LatencePredictionElevee
25      expr: histogram_quantile(0.95, rate(inflation_prediction_latency_seconds_bucket[5m])) > 10
26      for: 2m
27      labels:
28        severity: warning

```

Règles d'alertes (monitoring/alerts.yml) :



C21 : Gestion des incidents

Deux incidents documentés :

Incident 1 du 2026-07-18 : Erreur PySpark (JAVA_HOME)

- **Symptôme** : Failed to build wheel for pyspark a l'installation
- **Cause** : Java 21 installe mais PATH non rechargée dans la session VS Code en cours
- **Résolution** : fermeture/réouverture de VS Code, PATH rechargée automatiquement
- **Correctif** : note ajoutée dans docs/specs_techniques.md (limites connues)

(Captures : C1_C21_erreur_pyspark_install.png, C21_installation_java21.png)

Incident 2 du 2026-07-25 : 403 Forbidden sur API modele (Docker)

- **Symptôme** : client Streamlit reçoit 403 avec la bonne clé API
- **Cause** : load_dotenv(override=True) dans auth.py écrasait la variable API_KEY injectée par Docker avec la valeur .env locale
- **Résolution** : override=False + bloc environment dans docker-compose.yml pour forcer POSTGRES_HOST=postgres et POSTGRES_PORT=5432 en contexte Docker
- **Commit correctif** : fix(C5/C19): corriger connexion PostgreSQL API dans Docker

(Postmortem : docs/incident_2026-07-25.md)

3. Bilan et limites assumées

Résultats techniques

Indicateur	Valeur
Tests automatisés	47/47
Couverture de code	93%
Pipelines CI/CD	3 (data, modele, app), tous verts
Catégories prédites	12 / 13 COICOP (exclusion catégorie 12 documentée)
MAE moyenne Prophet	1.43 pts IPC (1.31% MAPE)
Volume données	3,68 M lignes dans inflation_unified
Sources intégrées	5 (INSEE, BCE, Eurostat, DATAGOUV, OpenFoodFacts)

Limites assumées

DATAGOUV rebase 2025 : les valeurs DATAGOUV (base 2025=100) ne sont pas comparables en valeur absolue avec INSEE/ECB/Eurostat (base 2015=100). Les tendances relatives restent valides. La colonne base_ref permet à l'application d'informer l'utilisateur.

Horizon Prophet : la précision se dégrade au-delà de 12 mois et sur les catégories très volatiles (Energie, communications). L'application limite le slider à 36 mois avec avertissement.

OpenFoodFacts non visualise : collecte (~5 k lignes de prix alimentaires) mais non intégré dans les prédictions. Volume insuffisant et couverture géographique partielle.

Réentraînement manuel : les modèles Prophet sont réentraînés à la demande (python model/train.py), pas automatiquement à chaque nouvelle donnée INSEE mensuelle. Un cron job ou un trigger webhook serait la prochaine étape.

Perspectives

- Automatiser le réentraînement mensuel (trigger sur nouvelles données INSEE)
- Ajouter la catégorie 12 (Biens et services divers) si la série BDM est identifiée
- Intégrer les prix OpenFoodFacts dans un modèle hybride indices + prix terrain
- Déployer sur un serveur distant (VPS ou cloud) pour une démo accessible sans Docker local